Dnd5E Implementation Breakdown

Summary: Next Steps: Write unit tests Integrate with Being class Update state schema Create more scenarios Add 5e SRD data integration.

Generated: 2026-01-12 05:36 UTC | **Ideas Extracted:** 47

What Happened

> Next Steps: Write unit tests Integrate with Being class Update state schema Create more scenarios Add 5e SRD data integration.

Design: All methods are @staticmethod - no state, pure functions.

> We successfully reverse-engineered the D&D 5e "physics engine" from multiple repositories and implemented it as the "biology" for WAFT Being agents. This document breaks down the critical findings, implementation decisions, and the architecture we built.

roll, iscritical = rolld20(advantage, disadvantage) total = roll + abilitymodifier + proficiencybonus hit = total >= targetac or iscritical.

Finding: Critical hits (Natural 20) are a separate boolean flag, not just a high number.

Rationale: Critical hits have special rules (always hit, double damage dice).

> Implementation: attackroll() returns (roll, iscritical) tuple.

Generated: 2026-01-11 Purpose: Comprehensive breakdown of D&D 5e implementation findings and architecture Status: Implementation Complete.

> These are the immutable laws of the simulation - we don't invent them, we implement the math found in the repositories.

Implementation: Equipment slots as optional strings: equippedweapon: Optional[str] = None equippedarmor: Optional[str] = None.

Relevance: This is perfect for LLMs. The AI outputs a text command, and our system parses and executes it. It creates a clean interface between the "Brain" (AI) and the "Body" (Game Engine).

Methods: abilitymodifier(score: int) -> int proficiencybonus(level: int) -> int calculateac(dexmodifier, armortype, armorbase) -> int spellsavedc(spellcasting_mod, proficiency) -> int.

> Methods: roll(expression: str) -> int attackroll(advantage, disadvantage) -> tuple[int, bool] rolldamage(dice_expression: str) -> int.

Methods: makeattackroll(attackmodifier, targetac, advantage, disadvantage) -> tuple[bool, bool] makesavingthrow(abilitymod, proficiency, isproficient, dc) -> bool applydamage(character, damage) -> tuple[int, bool] applyhealing(character, healing) -> int.

> Comprehensive docstrings for all functions: Parameter descriptions Return value descriptions Examples where helpful Formula explanations.

We didn't invent D&D mechanics - we extracted them from proven implementations. This saved weeks of design work.

We successfully reverse-engineered the D&D 5e physics engine and implemented it as a clean, well-structured module. The implementation follows best practices:.

✅ Input validation ✅ Error handling ✅ Type hints ✅ Comprehensive documentation ✅ Separation of concerns ✅ Testable design.

> The system is ready for integration with WAFT Beings and can serve as the "biology" for agent capabilities.

This is the heartbeat of D&D. Every stat (STR, INT, etc.) boils down to this single formula.

> Implementation: DnD5eStats.ability_modifier(score: int) -> int.

Implementation: DnD5eStats.calculateac(dexmodifier, armortype, armorbase).

This is the "Experience" curve. It scales with level, not stats.

Additional Details

Implementation: DnD5eStats.proficiency_bonus(level: int) -> int.

Implementation: DnD5eCombat.makeattackroll(attackmodifier, targetac, advantage, disadvantage).

How we structure the "Soul" of the agent (20.00_state.json).

> Future Implementation: Spell data structure with metadata fields.

Problem: Some source systems (like pixel games) use 4 stats, but D&D uses 6.

> Implementation: StatsAdapter.convert4to6(str, dex, int, con, charclass).

Future Implementation: Command parser for LLM-generated actions.

Future Implementation: Checksum validation for state files.

Error Handling: All methods wrapped in try/except with clear error messages.

> Purpose: Convert 4-stat systems to 6-stat D&D format.

Method: convert4to6(str, dex, int, con, charclass) -> Dict[str, int].

> Decision: Store ability scores (16 STR), calculate modifiers (+3) at runtime.

Implementation: Properties calculate modifiers on-demand:.

Decision: Store hp and max_hp separately, not just hp with a calculation.

Rationale: Max HP can change (level up, items), and we need to track both.

> Decision: Natural 20 is a separate boolean flag, not just a high number.

Decision: AC calculation has conditional logic based on armor type.

> Rationale: This is a core D&D 5e rule - heavy armor doesn't benefit from DEX.

Implementation: if/else logic in calculate_ac():.

Decision: Validate all inputs (ability scores 1-30, levels 1-20).

Implementation: Validation in __post_init__ and all calculation methods.

> Created an interactive scenario demonstrating the D&D 5e system in action.

Approach: Add optional dnd5e_character: Optional[DnD5eCharacter] field to Being class.

> Status: Implementation Complete Date: 2026-01-11 Version: 1.0.0.

Content Breakdown

Type	Count
Decisions	6
Insights	4
Actions	21
Concepts	14
Questions	2